

→ Курси

Master of Science: Artificial Intelligence and Machine Learning

→

Пошук

🔍

📈

📅

🔔

Міні-курси





→

No...

▼

- ✓ Огляд ди
- ✓ Тема 1.
- ✓ Наукова
- ✓ Тема 2.
- ✓ Тема 3.
- ✓ Тема 4.
- ✓ Тема 5.
- ✓ Тема 6.
- ✓ Наукова
- ✓ Тема 7.
- ✓ Завдання
- ✓ Тема 8.
- ✓ Тема 9.
- ✓ **Завдання**
- ✓ Тема 10.
- ✓ Фінальне

Завдання 3. Граф знань для рекомендаційно. системи

Вітаємо! Це ваше фінальне заняття з дисципліни «NoSQL та векторні бази даних».

Мета цього завдання: освоїти повний цикл роботи з графовою базою даних — від проєктування схеми та завантаження реальних даних до написання складних запитів і запуску алгоритмів аналізу графів. На практиці зрозуміти, де графова модель виграє у реляційної, а де програє.

Опис завдання

Ви будете рекомендаційний рушій на основі графа. Дані — реальний датасет фільмів з оцінками користувачів.

Датасет: [MovieLens 1M](#) — 1 мільйон оцінок від 6 040 користувачів для 3 883 фільмів, з жанрами та демографією. Зібраний GroupLens Research широко використовується в академічних дослідженнях рекомендаційних систем.

Архів містить три файли даних і README з детальним описом — прочитайте його до початку роботи:

- **movies.dat** — 3 883 фільми: унікальний ID, назва з роком випуску та список жанрів через | (18 жанрів: від Action і Comedy до Film-Noir і Children's).
- **users.dat** — 6 040 користувачів: стать, вікова група, код професії та поштовий індекс.
- **ratings.dat** — 1 000 209 оцінок за шкалою 1–5 з Unix-таймстемпом; кожен користувач оцінив не менше 20 фільмів.

Усі три файли використовують роздільник :: і кодування Latin-1 — це важливо при завантаженні (*детальніше в частині 2*). Завантажте архів і розпакуйте його.

Хід роботи:

1. Завантажити та розпакувати датасет MovieLens 1M, конвертувати .dat файли у CSV.
2. Розгорнути Neo4j через Docker або зареєструватися в AuraDB.
3. Спроекувати схему графа та обґрунтувати рішення — вузли, ребра, властивості.
4. Завантажити дані: створити індекси, завантажити вузли та ребра батчами.
5. Написати 6 Cypher-запитів зростаючої складності — від простих фільтрацій до рекомендацій і пошуку шляхів.
6. Знайти супервузли та пояснити їхній вплив на продуктивність.
7. Запустити три алгоритми GDS: PageRank, Louvain і Dijkstra — та змістовно інтерпретувати результати.
8. Порівняти графовий підхід з реляційним і зробити висновки.

Структура репозиторію

```
.
├── docker-compose.yml      # локальний запуск Neo4j
├── convert.py              # конвертація .dat → .csv
├── import/                 # CSV-файли для завантаження (не
    комітьте самі .dat)
│   ├── movies.csv
│   ├── users.csv
│   └── ratings.csv
├── queries/
│   └── part2_load.cypher    # створення індексів і завантаження
    даних
│   ├── part3.cypher        # всі запити частини 3
│   ├── part4_supernodes.cypher
│   └── part5_gds.cypher
└── README.md               # відповіді на всі питання + скриншоти
```

Детальніше по кожному кроку дивіться в описі ДЗ.

До наступного модуля